

The University Library & Staff System

The Goal: Build a backend system that manages the people in a university library. You must handle data processing from a file, create a class hierarchy, and use recursion to calculate the total salary budget of the library hierarchy.

1. The Scenario

You are the Lead Developer for a University Library. The system needs to track three types of people:

1. **Members:** Students who pay fees and borrow books.
 2. **Employees:** Staff who work there and earn a salary.
 3. **Student Workers:** The complex case! These are students who work at the library. They have to pay fees (like a Member) but also earn a salary (like an Employee).
-

2. The Class Architecture (OOP)

You must implement the following hierarchy.

Level 1: The Base Class

- **Class Person**
 - Attributes: `name`, `id_number`.
 - Method: `__str__` to return a summary of the person.

Level 2: The Parent Classes

- **Class Member (Inherits Person)**
 - Attributes: `membership_fees_owed` (Float).
 - Method: `pay_fees(amount)`: Reduces the fees owed.
- **Class Employee (Inherits Person)**
 - Attributes: `salary`, `subordinates` (List of other Employees).
 - Method: `add_subordinate(employee)`: Adds another employee to their list.
 - **Method (Recursive):** `get_total_hierarchy_cost()`: Returns this employee's salary + the `get_total_hierarchy_cost()` of all their subordinates.

Level 3: The Child Class (Multiple Inheritance)

- **Class `StudentWorker` (Inherits `Member` AND `Employee`)**
 - *Concept:* This class represents a student who works part-time.
 - *Attributes:* Inherits all attributes from both parents.
 - *Logic:* When initialized, they should be able to hold both a `salary` and `membership_fees_owed`.
 - *Polymorphism:* Create a method `net_income()` which returns `salary - membership_fees_owed`.
-

3. The Data (File I/O)

Create a file named `library_people.txt`. (Format: *Type, ID, Name, Value1, Value2/ReportsToID*)

- **Type M (Member):** ID, Name, FeesOwed, (None)
- **Type E (Employee):** ID, Name, Salary, ReportsToID
- **Type S (StudentWorker):** ID, Name, Salary, FeesOwed, ReportsToID

Code snippet

```
None
E, 1, Head Librarian, 80000, 0
E, 2, Section Manager, 50000, 1
M, 3, Regular Student, 500, 0
S, 4, Working Student, 12000, 500, 2
S, 5, Archive Intern, 10000, 200, 2
```

4. Technical Requirements (The Code Logic)

Part A: File Reading & Object Creation

1. Create a dictionary or list to store created objects so you can link them later.
2. Read the file line by line.
3. **Exception Handling:**
 - Use a `try/except` block to parse the numbers (Salary/Fees). If the file contains text where a number should be (e.g., "TenThousand"), print "Error processing line X" and skip it.
4. **Instantiation:** Based on the first letter (`E`, `M`, or `S`), create the correct Object.

Part B: Logic & Linking

- After creating the objects, you have to link the Employees.
- The `StudentWorker` (ID 4) reports to `Section Manager` (ID 2). You must find Object ID 2 and add Object ID 4 to their `subordinates` list.

Part C: Recursion (The Budget)

- You must be able to call `get_total_hierarchy_cost()` on the **Head Librarian**.
- The Head Librarian adds their salary (\$80k).
- Then loops through subordinates -> calls the method on Section Manager.
- Section Manager adds their salary (\$50k).
- Then loops through subordinates -> calls method on Working Student and Archive Intern.
- **Result:** It sums up the entire tree automatically.

Part D: Output

- Write a file `payroll_report.txt` that shows the structure and the total calculated cost.

Hint for the Multiple Inheritance Class

Python

```
class StudentWorker(Member, Employee):
    def __init__(self, id_num, name, salary, fees_owed):
        # Explicitly initialize both parents
        # Note: Logic may vary depending on how you set up parent
    __init__
        Employee.__init__(self, id_num, name, salary)
        Member.__init__(self, id_num, name, fees_owed)
```

(Note: In Python 3, `super()` can handle this if the parents are designed cooperatively, but explicit calls like above are often easier to understand conceptually.)